

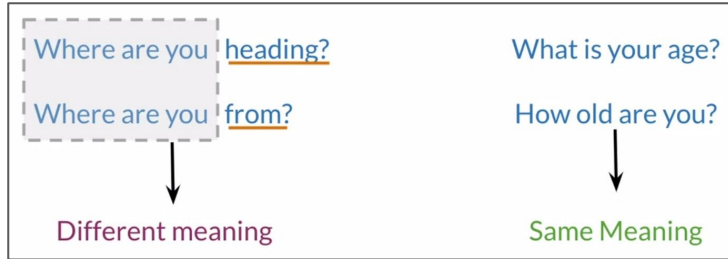
Word Embeddings and Vector Spaces

NLP - Week#07(2)

Vector Space Models

- Vector space models help you identify whether the first pair of questions or the second pair are similar in meaning even if they do not share the same words. Similarity identification has applications in question answering, paraphrasing, and summarization.
- Vector space models also allow you to capture dependencies between words by representing words as vectors, referred to as word vectors/vector representations of words/word embeddings.
- With vector space models, you can capture relationships among different sets of words. Vector space models are used in information extraction to answer questions in the style of who, what, where, how, etc. – which have applications in information extraction, machine translation and chatbots.

Vector Space Models



“You shall know a word by the company it keeps”

Firth, 1957



(Firth, J. R. 1957:11)

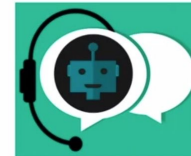
- You eat cereal from a bowl
- You buy something and someone else sells it



Information Extraction



Machine Translation



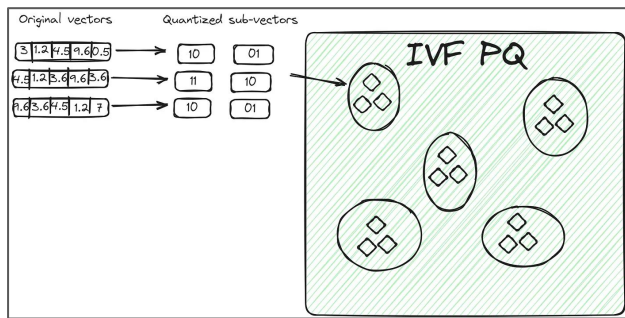
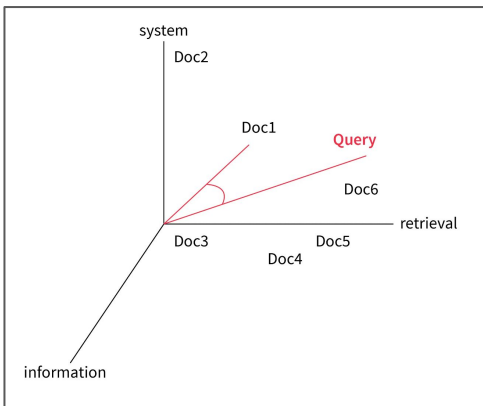
Chatbots

Vector Space Models

- Vector space models enable representing words in text documents like tweets, articles, queries, or more generally, an object that contains text, as a vector.
- Since these word vectors capture relative meaning by identifying the context of the word, they lend themselves to applications where similarity identification is of essence. This includes the fields of information retrieval, indexing, relevancy ranking, information filtering etc.
 - For e.g., when you look up a query online, the algorithm relies on the above concepts to give you back your results.
- NLP fundamental concept: "You shall know a word by the company it keeps" - Firth, 1957.
- Applications include:
 - Information Extraction
 - Machine Translation
 - Chatbots

Examples

- Information Retrieval (IR):
 - If you search “doctor salary”, the system can also retrieve pages with “physician income” — because doctor $\approx$ physician, salary $\approx$ income in the vector space.
- Indexing
 - Indexing means organizing text data so that it can be quickly searched.
 - When indexing uses word vectors, it can group related words together, so the search system understands relationships like “car” $\approx$ “automobile”.
- Relevancy Ranking
 - Once the system finds multiple results, it has to rank them — which one is most relevant to your query?
 - Word vectors help measure how semantically close a document is to your query words.
 - The more similar the meanings, the higher the rank.
- Information Filtering
 - This is about showing or hiding information based on user preferences.
 - For example, in a news or recommendation system, word vectors help filter articles that are similar in meaning to topics you’ve liked before — even if the words are different.



Word by Word Design

Number of times they *occur together within a certain distance k*

I like simple data

I prefer simple raw data

$k=2$

	simple	raw	like	I
data	2	1	1	0

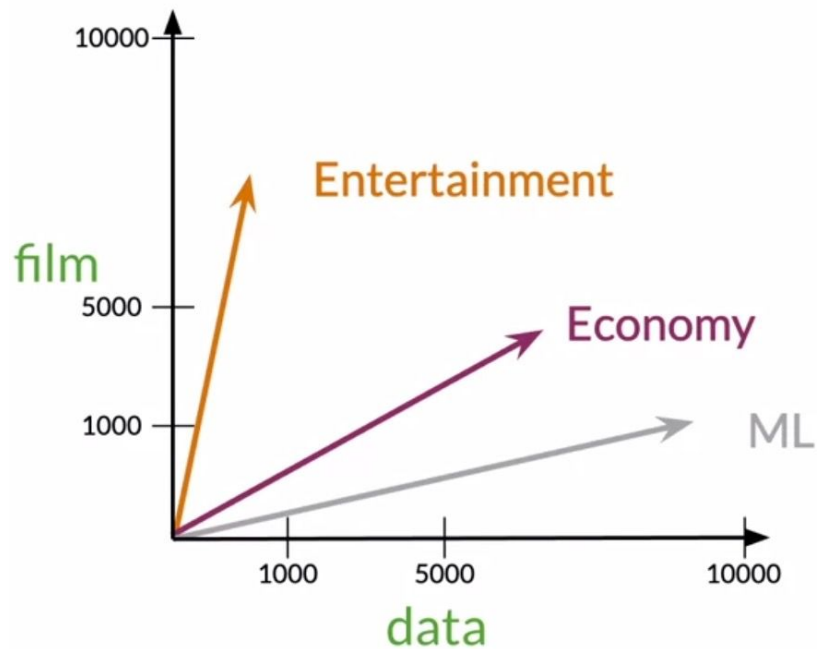
Word by Document Design

- You can get vector spaces using two different designs: word by word and word by document.
 - Word by Word: count the co-occurrence of words, i.e., the number of times two words occur together within a certain distance k .
 - Word by Document: count the number of times a word occurs within each category of documents.
- In vector spaces, you can determine the “closeness” or relationships between different types of documents using similarity.

Number of times a word *occurs within a certain category*

	Entertainment	Economy	Machine Learning
data	500	6620	9320
film	7000	4000	1000

Word by Document Design

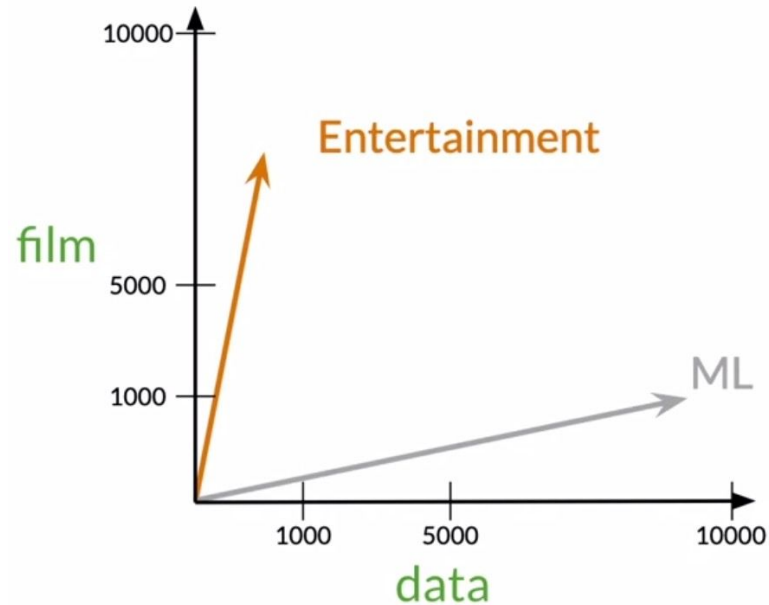


	Entertainment	Economy	ML
data	500	6620	9320
film	7000	4000	1000

Measures of “similarity:”
Angle
Distance

Euclidean Distance

Euclidean distance



Corpus **A**: (500,7000)

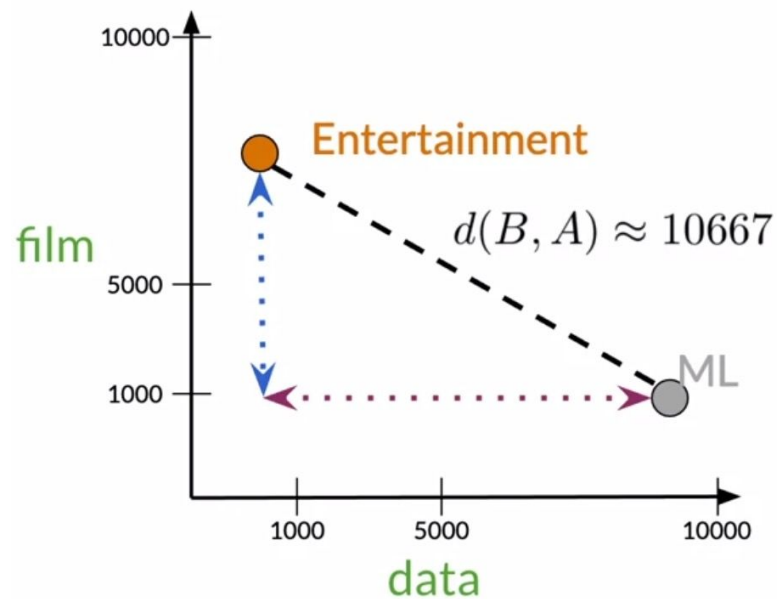


Corpus **B**: (9320,1000)

Euclidean Distance

- Euclidean distance is the length of the straight line that connects any two points, i.e., two vectors in vector space.
- To get the Euclidean distance, calculate the norm of the difference between the vectors being compared. By using this metric, you can get a sense of how similar two documents or words are.
- Euclidean distance can however be misleading when the corpora have different sizes.

Euclidean distance



Corpus **A**: (500,7000)



Corpus **B**: (9320,1000)

$$d(B, A) = \sqrt{\underbrace{(B_1 - A_1)^2}_{\text{purple}}} + \underbrace{(B_2 - A_2)^2}_{\text{blue}}$$

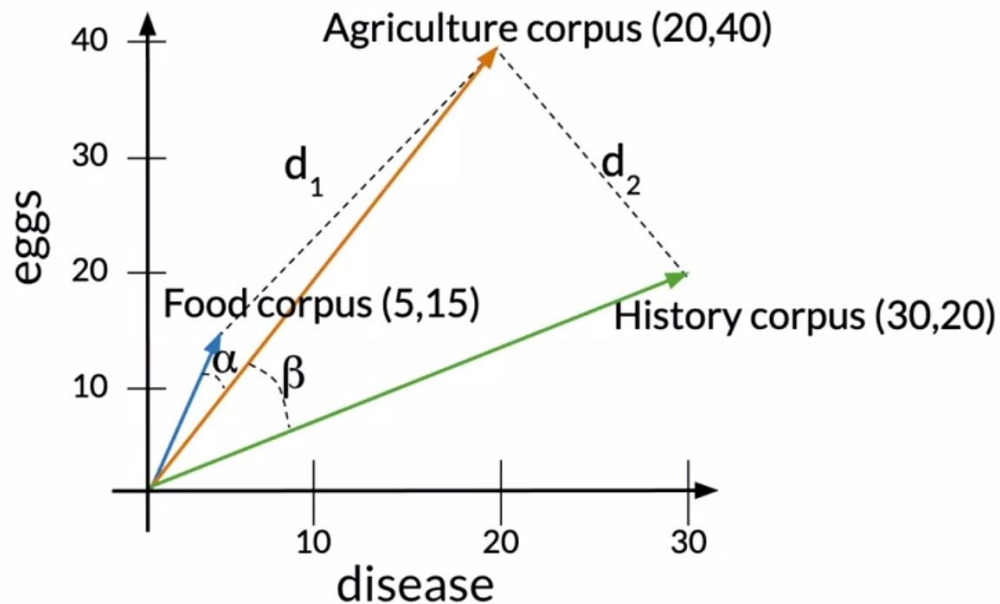
$$c^2 = a^2 + b^2$$

$$d(B, A) = \sqrt{(8820)^2 + (-6000)^2}$$

Cosine Similarity

- The cosine similarity between a pair of vectors is proportional to the similarity between the directions of the vectors that you are comparing.
- (+) The intuition behind the use of the cosine similarity as a metric to compare the similarity between two vector representations is that it isn't biased by the size difference between the representations. This is the main advantage of the cosine similarity metric over euclidean distance.

Cosine Similarity

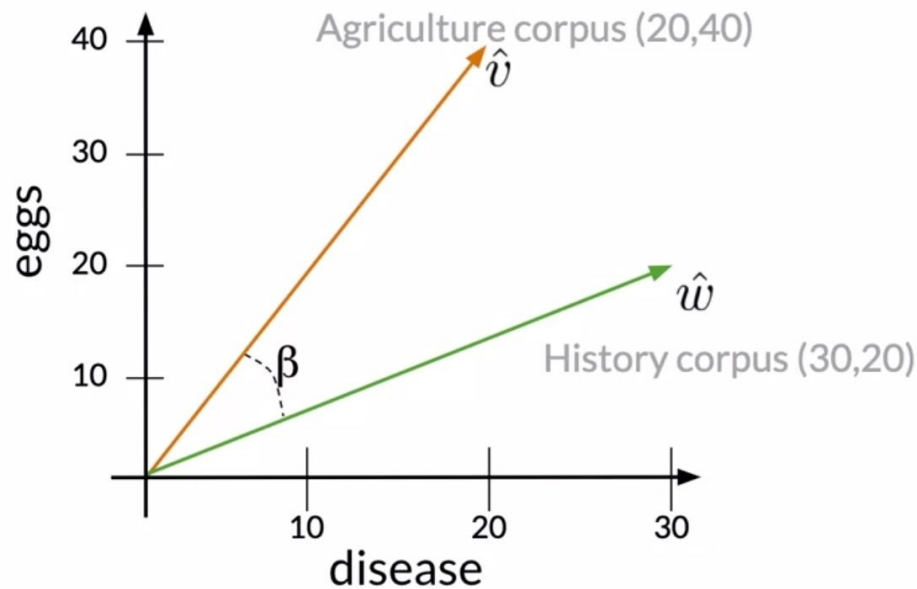


Euclidean distance: $d_2 < d_1$

Angles comparison: $\beta > \alpha$

The cosine of the angle
between the vectors

Cosine Similarity

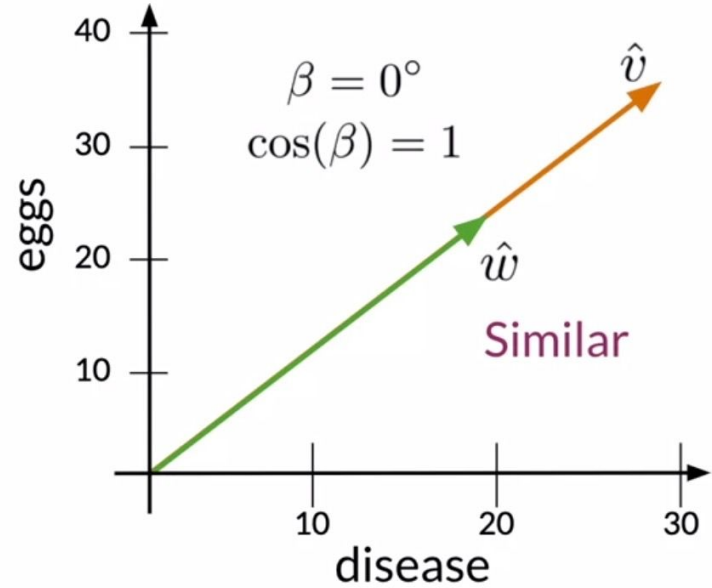
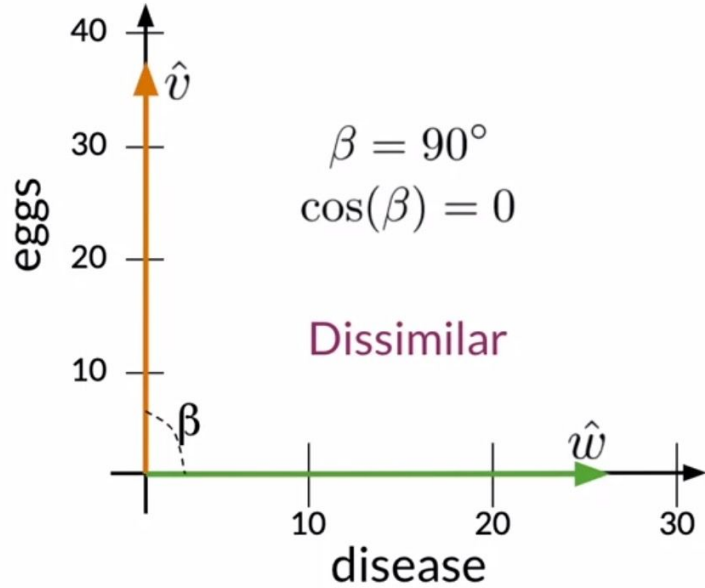


$$\hat{v} \cdot \hat{w} = \|\hat{v}\| \|\hat{w}\| \cos(\beta)$$

$$\cos(\beta) = \frac{\hat{v} \cdot \hat{w}}{\|\hat{v}\| \|\hat{w}\|}$$

$$= \frac{(20 \times 30) + (40 \times 20)}{\sqrt{20^2 + 40^2} \times \sqrt{30^2 + 20^2}} \\ = 0.87$$

Cosine Similarity



Word Manipulation in Vector Space

- Let's perform vector manipulation using simple vector arithmetic, i.e., by adding vectors and subtracting vectors. We'll apply the idea of manipulate vector representations to the task of inferring unknown relationships among words, such as predicting the countries of certain capitals.
- Vector manipulation involves using some simple vector algebra to carry out such tasks.



USA



Washington
DC

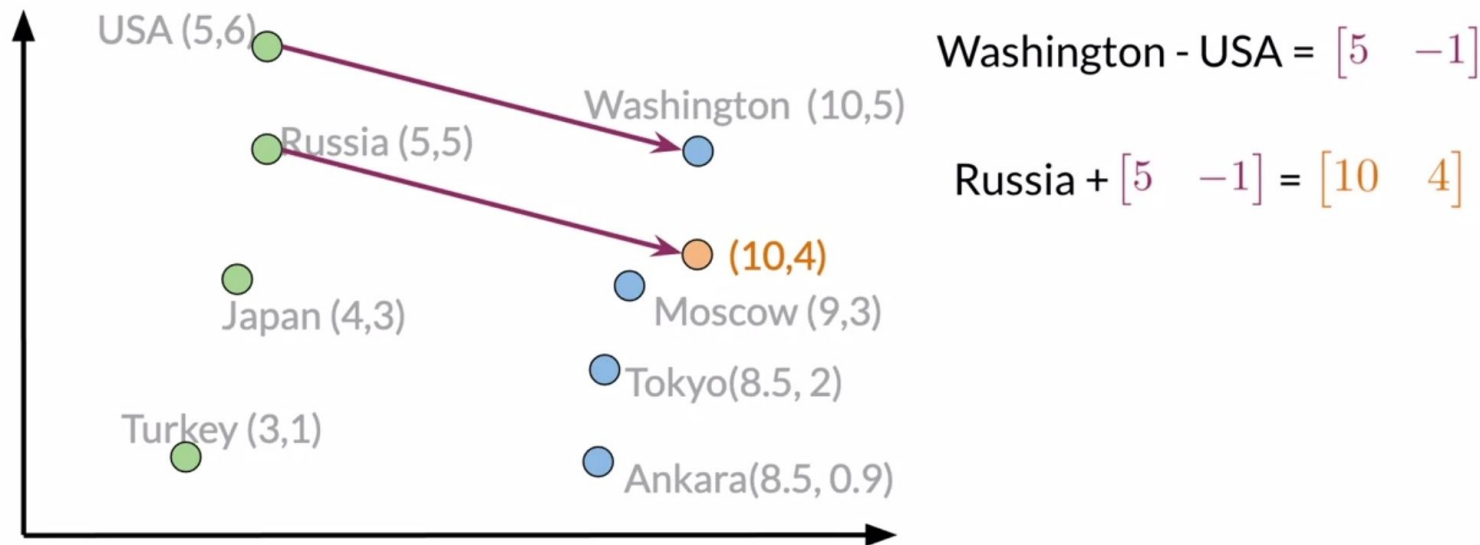


Russia



?

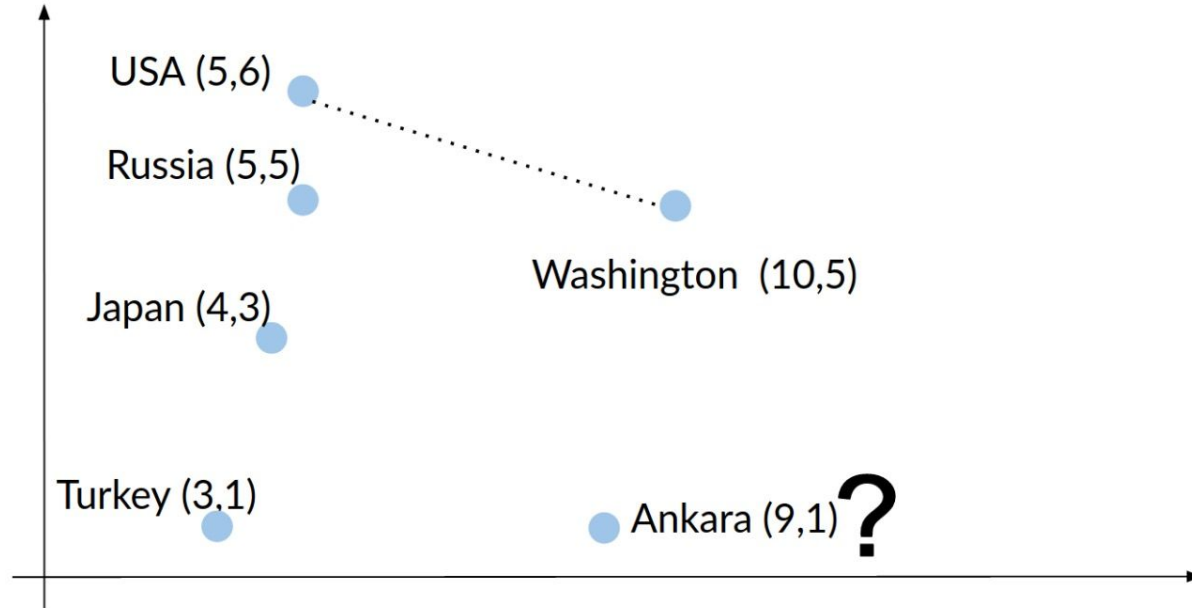
Word Manipulation in Vector Space



[Mikolov et al, 2013, Distributed Representations of Words and Phrases and their Compositionality]

Try it

Using the same method, let's predict the country whose capital is Ankara.



Class Activity - Reading Task

<https://mehmetozkaya.medium.com/how-vector-databases-work-vector-creation-in-dexing-and-search-0da2226d04dc>

Reading

https://www.microsoft.com/en-us/research/wp-content/uploads/2021/11/SPANN_finalversion1.pdf

SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search

PCA

- PCA is an algorithm used for dimensionality reduction that can find uncorrelated features in your data. It's helpful for visualizing your data to check if your representation is capturing relationships among words.
- After obtaining uncorrelated features, PCA projects your data representation to a lower dimension while retaining as much information as possible.

PCA - Principal Component Analysis

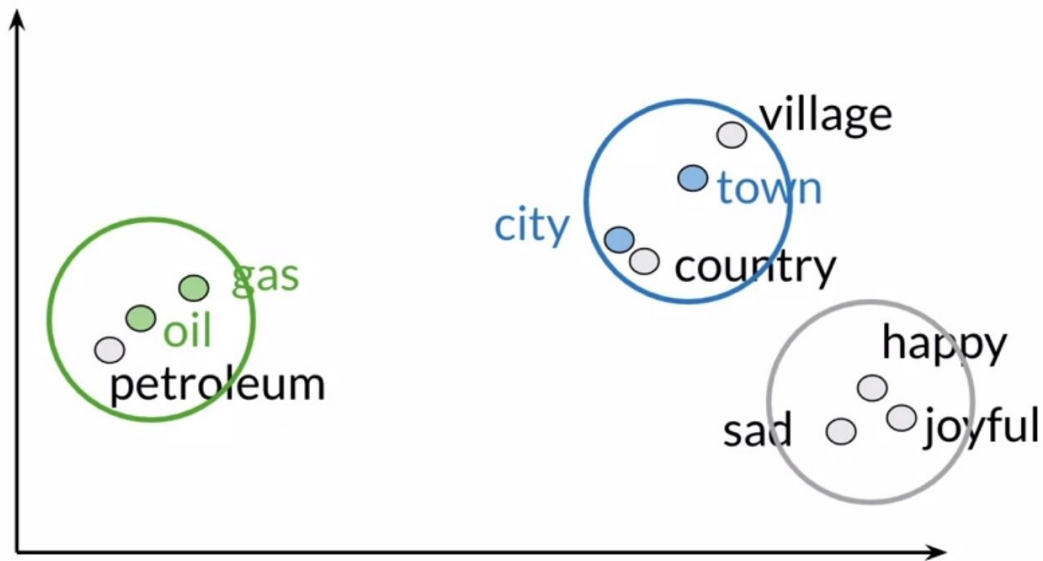
Visualization of Word Vectors

PCA enables you to get a representation in a vector space with fewer dimensions. To ease the process of visualization of data, you can get a reduced representation with fewer features (say, 2 or 3).

	$d > 2$				$d = 2$	
oil	0.20	...	0.10	PCA →	oil	2.30 21.2
gas	2.10	...	3.40		gas	1.56 19.3
city	9.30	...	52.1		city	13.4 34.1
town	6.20	...	34.3		town	15.6 29.8

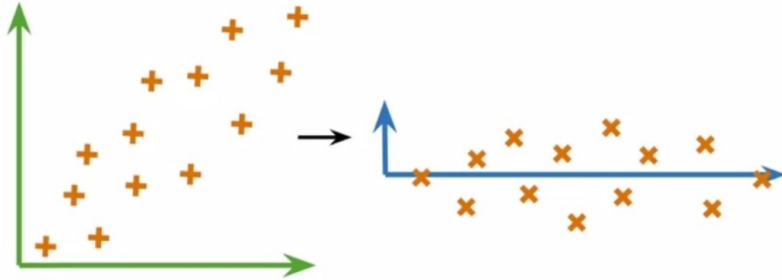
PCA

Applying PCA on your data can get you a 2D representation, which you can then use to plot an XY visual of your words.



PCA Algo

PCA algorithm



Mean Normalize Data $x_i = \frac{x_i - \mu_{x_i}}{\sigma_{x_i}}$

Get Covariance Matrix Σ

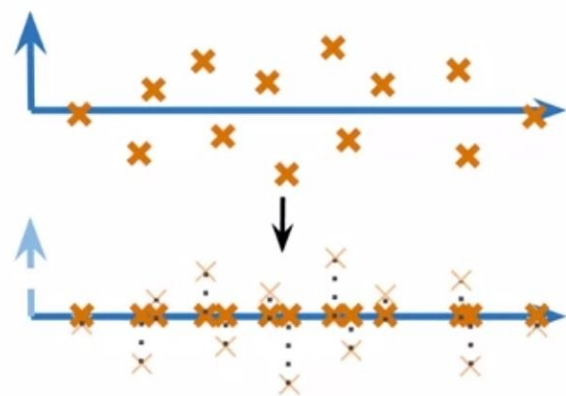
Perform SVD $SVD(\Sigma)$

Eigenvectors



Eigenvalues





Eigenvectors

Eigenvalues

$$\begin{pmatrix} | & | & | & | \\ U & \dots & & \\ | & | & | & | \end{pmatrix}$$

$$\begin{pmatrix} \blacksquare & & & \\ & \blacksquare & & \\ & & \blacksquare & \\ & & & \blacksquare \\ & & & & S \\ & & & & & \ddots \\ & & & & & & \blacksquare \end{pmatrix}$$

Dot Product to
Project Data

$$X' = XU[:, 0 : 2]$$

Percentage of
Retained Variance

$$\frac{\sum_{i=0}^1 S_{ii}}{\sum_{j=0}^d S_{jj}}$$

PCA

- Goals
 - (+) PCA enables high-dimensional data visualization by carrying out dimensionality reduction to represent data in a reduced space of uncorrelated variables, say on a 2D plane.
- Process
 - At a top level, PCA involves the following steps:
 1. Get uncorrelated features: Using the word representations in your original vector space, get uncorrelated features.
 2. Project uncorrelated features to a lower-dimensional vector space: Reduce the dimensionality of your data by projecting your data to a vector space with the desired number of features, while trying to retain as much information as possible from your original embedding.
- Concepts
 - PCA is based on the Singular Value Decomposition (SVD) of the covariance matrix of the original high dimensional dataset.
 - Eigenvectors from the “decomposed” covariance matrix contain the principal components, which are encoded as a rotation matrix that contains the directions of the uncorrelated features.
 - Eigenvalues associated with the aforementioned eigenvectors represent the “explained variance” of each principal component, i.e., variance of your data on the new features. In other words, they signal the amount of information retained by each new feature.
- Implementation
 - Get uncorrelated features
 1. Mean normalize data by operating on each feature separately (across the entire dataset).
 2. Compute the covariance matrix.
 3. Perform SVD on the covariance matrix to get the eigenvectors and eigenvalues.
 - Project uncorrelated features to a lower-dimensional vector space
 1. The dot product of your word embeddings and the eigenvectors matrix will project your data onto a new vector space of the desired dimension.
 2. Compute the amount of retained variance (in %) using the eigenvalues.